

Giorno 14 — Sicurezza REST completa e avvio advanced mappings

Esercizio 1 — Sicurezza con JDBC e BCrypt

Migra la configurazione della sicurezza da utenti in-memory a utenti memorizzati nel database, e implementa la cifratura sicura delle password con BCrypt.

Cosa fare:

- Crea nel database due tabelle per la sicurezza Spring: users (username VARCHAR, password VARCHAR, enabled BOOLEAN) e authorities (username VARCHAR, authority VARCHAR). Popolale con almeno 2 utenti: admin/ROLE_ADMIN e user/ROLE_USER.
- Modifica SecurityConfig per usare JdbcUserDetailsManager al posto degli utenti in-memory. Configura il datasource. Riavvia e verifica che il login funzioni con gli utenti del database.
- Installa BCryptPasswordEncoder come bean nel SecurityConfig. Aggiorna le password nel database codificandole con BCrypt (puoi usare un piccolo test main per generare gli hash). Aggiorna il PasswordEncoder nel config.
- Verifica che il login funzioni ancora con le password BCrypt. Prova a leggere la colonna password nel database: non deve essere più leggibile in chiaro.
- Aggiungi un endpoint POST /api/auth/registrazione che accetta {username, password, ruolo} e crea un nuovo utente nel database con la password cifrata. Gestisci il caso in cui l'username esista già (risposta 409).
- Crea tabelle personalizzate invece delle tabelle predefinite di Spring (es. utente e ruolo con colonne diverse). Configura Spring Security per usare le tue tabelle con query SQL personalizzate.

Consegna: Dimostra il flusso completo: registra un nuovo utente via API, poi usa le sue credenziali per autenticarti su un endpoint protetto. Documenta le query SQL usate per creare le tabelle.

Esercizio 2 — Prima relazione @OneToOne

Inizia a lavorare con i mapping avanzati JPA creando una relazione One-to-One tra due entità. Modellerai la relazione tra uno Studente e il suo Profilo personale.

Cosa fare:

- Crea un'entità ProfiloStudente con: id, bio (TEXT), linkedin (String), foto_url (String), data_creazione (LocalDateTime). Annota con @Entity.
- Aggiungi all'entità Studente un campo ProfiloStudente profilo. Configuralo con @OneToOne e @JoinColumn(name="profilo_id") per creare la foreign key.
- Aggiungi alla relazione cascade=CascadeType.ALL: questo farà sì che quando salvi uno Studente con un Profilo associato, anche il Profilo venga salvato automaticamente.
- Nel DataLoader, crea almeno 2 studenti con profilo e 1 senza. Verifica a DB la foreign key sia presente nella tabella STUDENTI.
- Crea un endpoint GET /api/studenti/{id}/profilo che restituisce il profilo dello studente (o 404 se non ha un profilo).
- Implementa la relazione bidirezionale: aggiungi in ProfiloStudente un campo @OneToOne(mappedBy="profilo") Studente studente. Fai attenzione alla serializzazione JSON circolare: usa @JsonIgnore su uno dei lati.
- Testa la cancellazione: elimina uno studente con il profilo associato e verifica (grazie a CascadeType.ALL) che anche il profilo venga eliminato.

Consegna: Il progetto deve essere funzionante. Carica il codice su GitHub.